Лабораторная работа № 12

Программирование в командном процессоре ОС UNIX. Командные файлы

Чжан Вэньцзюнь

РУДН

2026-10-20

Содержание I

1. Информация
2. Теоретические сведения
3. Выполнение работы
4. Заключение

Раздел 1

Информация

Докладчик

- Чжан Вэньцзюнь
- зачётная книжка № 1132259051
- Российский университет дружбы народов им. П. Лумумбы

Цель работы

- Изучить основы программирования в оболочке ОС UNIX/Linux
- Научиться писать небольшие командные файлы
- Освоить переменные, параметры и управляющие конструкции bash

Задание

1. Скрипт резервной копии самого себя в `~/backup` (`tar`/`zip`/`bzip2`)
2. Скрипт обработки произвольного числа аргументов (в том числе > 10)
3. Аналог команды `ls` без использования `ls` и `dir`
4. Скрипт подсчёта файлов заданного формата в указанной директории

Раздел 2

Теоретические сведения

Командные оболочки

- `sh` — оболочка Борна, базовый набор
- `csh` — C-подобный синтаксис, история команд
- `ksh` — оболочка Корна
- `bash` — Bourne Again Shell, свойства `csh` и `ksh`

Переменные и специальные параметры

Переменная	Назначение
$0	имя запущенного файла
$1...${10}	позиционные параметры
$#	число параметров
$*, $@	все параметры
$?	код возврата
$$	PID оболочки
$!	PID фонового процесса

Управляющие конструкции

```
if [ -d "$DIR" ]; then ...; fi
for f in "$@"; do ...; done
while [ ... ]; do ...; done
case "$x" in *.txt) ...;; esac
```

Раздел 3

Выполнение работы

1. Резервная копия самого себя

```
#!/bin/bash
DEST="$HOME/backup"
mkdir -p "$DEST"
STAMP=$(date +%Y%m%d-%H%M%S)
tar -czf "$DEST/$(basename "$0").$STAMP.tar.gz" "$0"
```

- `$0` — путь к самому скрипту
- `tar -czf` — создание архива с gzip-сжатием
- архив проверен: `tar -tzf ~/backup/*.tar.gz`

```
vzhang@vzhang:~/work/os/lab10$ cat backup.sh
#!/bin/bash
# резервная копия собственного исходного кода в ~/backup (tar)
DEST="$HOME/backup"
mkdir -p "$DEST"
NAME="$(basename "$0")"
STAMP=$(date +%Y%m%d-%H%M%S)
tar -czf "$DEST/${NAME}.${STAMP}.tar.gz" "$0"
echo "Архив создан: $DEST/${NAME}.${STAMP}.tar.gz"
ls -lh "$DEST" | tail -3

vzhang@vzhang:~/work/os/lab10$ ./backup.sh
```

2. Обработка произвольного числа аргументов

```
for a in "$@"; do echo "[illegible]: $a"; done
shift 2
echo "[illegible]: $#"
```

- "$@" корректно перебирает любое число аргументов
- $# — количество, `shift n` — сдвиг списка

3. Аналог команды ls

```
for f in "$DIR"/* "$DIR"/.[!.]*; do
  [ -e "$f" ] || continue
 printf '%-48s %s\n' "$(basename "$f")" "$(stat -c '%A %U:%G %6s %y
done
```

- Не используются `ls` и `dir`
- Права доступа — через `stat %A`
- Выводятся и скрытые файлы (`.[!.]*`)

4. Подсчёт файлов заданного формата

```
EXT="${1:?[illegible]}"
DIR="${2:-.}"
n=0
for f in "$DIR"/*."$EXT"; do [ -f "$f" ] && n=$((n+1)); done
echo "[illegible] .$EXT [illegible] $DIR: $n"
```

- `${1:?}` — обязательный аргумент с сообщением
- `${2:-.}` — значение по умолчанию
- результат: `.sh` — 4 файла, `.png` — 3 файла

```
vzhang@vzhang:~/work/os/lab10$ cat myls.sh
#!/bin/bash
# аналог ls без использования команд ls и dir
DIR="${1:-.}"
[ -d "$DIR" ] || { echo "Не каталог: $DIR" >&2; exit 1; }
echo "Каталог: $(cd "$DIR" && pwd)"
for f in "$DIR"/* "$DIR"/.[!.]*; do
  [ -e "$f" ] || continue
  printf '%-48s %s\n' "$(basename "$f")" "$(stat -c '%A %U:%G %6s %y' "$f")"
done

vzhang@vzhang:~/work/os/lab10$ ./myls.sh ~/work/os/lab10
```

Раздел 4

Заключение

Выводы

- Изучены переменные, подстановка и арифметика в `bash`
- Освоена передача параметров (`$0`, `$1`, `$#`, `$@`, `shift`)
- Изучены управляющие конструкции (`if`, `for`, `while`, `case`)
- Написаны и проверены 4 командных файла
- Освоены `tar`, `stat`, шаблоны имён и экранирование

Спасибо за внимание!

Вопросы?